

2400-DS1AL # Algorithms for Data Science

[Kokpit](#) / [Moje kursy](#) / [2400-DS1AL#2025L#277553](#) / [Lecture 6 & 7](#) / [Lecture notes](#)

Lecture notes

Credits: dr Krzysztof Fleszar

Lecture 5: Sorting

Now, we will be talking about the problem of sorting:

INPUT: Array $a[0..n-1]$

OUTPUT: Array $a'[0..n-1]$ such that a and a' have the same elements and a' is sorted non-decreasingly, that is, $a'[0] \leq a'[1] \leq \dots \leq a'[n-1]$.

Although we often need to sort something in our programs, we rarely need to actually implement a sorting algorithm as all popular programming languages include sorting functions in their standard libraries. However, it is still good to know the theory. On one hand, in order to decide which sorting algorithm is best suited for our case. Different sorting algorithms differ not only in their running times but also in additional properties that we might desire to have. On the other hand, there are situations where our program will run faster if we implement our own sorting algorithm.

Furthermore, sorting algorithms are a good showcase of various basic techniques used throughout algorithmics, from ones we have already learned (counting, divide and conquer) to new ones (data structures).

Some Theory First...

A sorting algorithm is called *based on comparisons* if it only compares the elements to be sorted without looking at their actual value.

Theorem: Any sorting algorithm based on comparison requires $\Omega(n \log n)$ comparisons to sort an array of length n .

Proof (sketch). This could be viewed as a puzzle: we have n coins of different weights and we want to order them from lightest to heaviest. We can compare two coins using the scales. How to do this if we want to use the scales the smallest possible number of times? As an example, let's try $n=5$. The coins can be ordered in $5! = 120$ possible ways. After one comparison, 60 possible ways are left (as in one half of the 120 ways, one of the two compared coins is heavier, whereas in the other half the other coin is heavier). After the second comparison, at least 30 possibilities still remain in the worst case no matter which two coins we compared (as the 60 possible ways are split over the two possible outcomes of the comparison, which implies that for the "worse" of the two outcomes at least half of the possible ways remain). By continuing this process ($120 \rightarrow 60 \rightarrow 30 \rightarrow 15 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$), we learn that at least seven comparisons are necessary in the worst case. In general, we need at least $\log_2(n!)$ comparisons, which is $\Omega(n \log n)$ (because $n! > (n/2)^{n/2}$).

Classic Sorting Algorithms

In the following, we briefly discuss the general idea of several classic sorting algorithms. For a pictorial and intuitive presentation, see the animations on the lecture slides. There you also find an overview table comparing the algorithms. For more details, you can take a look in the [CLRS book](#) or Internet sources such as Wikipedia. On the web, you also find movies illustrating the execution of sorting algorithms; some of them in a very creative way

Insertion Sort

Insertion sort works as follows. We add the first, second, third, fourth, etc. element of the input array to the output array always inserting it such that the added elements in the output array are sorted. One can implement insertion sort even such that the output is directly written into the input array.

Unfortunately, as we already know, whenever we insert a new element into an array on some position i , we first need to make room for it by pushing all the elements at position i or greater one step to the right. The running time for this pushing is proportional to the number of the elements that we move. In the worst case, when we want to add an element to the first position, we have to move all the elements that are currently in the array. This makes the time complexity of insertion sort $\Theta(n^2)$.

The time bound cannot be lowered by using binary search to find the right position to add an element: though now we would perform only $O(n \log n)$ comparisons in total, we still need to move $\Theta(n^2)$ times some element in the array. For this reason, we usually do not use binary search when implementing insertion sort—it does not make the overall algorithm faster; in contrary, it makes the algorithm even slower in some special cases (e.g. when the array is already sorted)!

Insertion sort has time complexity $\Theta(n^2)$, memory complexity $O(1)$ (here and in the following, we do not count the input and output arrays to the memory complexity), and is *stable* (a sorting algorithm is stable if any two equal elements remain in the same order as in the input); furthermore, insertion sort is fast ($O(n)$) when the input is already sorted.

Merge Sort

This sorting algorithm is based on the divide and conquer technique: split the array into two halves, sort them, and merge them.

The time complexity is always $\Theta(n \log n)$ and the memory complexity $O(n)$. Merge sort is stable.

Quick Sort

This algorithm is also based on divide and conquer, but it works in a different way than merge sort. We choose an element to be the *pivot* (for instance, the first element), move elements smaller than the pivot to the left, and larger than the pivot to the right. Then we sort both parts using the same algorithm.

There are bad cases where quick sort runs in time $\Theta(n^2)$ (when one of the smallest/largest elements is always chosen to be the pivot), but on the average it runs very quickly ($O(n \log n)$). If we take the pivot randomly, the algorithm will run very quickly with very high probability, so in this particular case, average time complexity is practically more useful than the pessimistic one.

The memory complexity is usually $O(\log n)$ but it can be $O(n)$ in the worst case. This is because, when a function calls itself recursively, the computer needs to remember the previous call (so-called *recursion stack*); therefore, in the worst case above, the computer will have to remember n calls on the recursion stack. It is possible to do it more cleverly and use $O(\log n)$ memory in the worst case—use recursion for the smallest part, and then solve the bigger part without using a recursive call.

Heap Sort

Heap sort is interesting, because it is the first algorithm in our lecture that is based on a nontrivial **data structure**.

Data structures are ways of arranging data. In most data structures there is some important property that defines how data is arranged. This property has to be satisfied after every operation (this property is called the *invariant* of the structure—similarly to loop invariants, which must be satisfied after every iteration of the loop). So far we have seen two basic data structures: unsorted array (no invariant) and sorted array (the invariant here is that the array should be sorted after every operation).

In an unsorted array, we can add a new element very quickly (just add it at the end), but it takes very long to find anything. In a sorted array, we can find every given element very quickly using binary search (the invariant helps us); however, as we have seen in insertion sort, adding a new element takes time (we need to spend extra time to keep the invariant satisfied). This is a common tradeoff in algorithmics (as well as in real life): it takes more time to make the data more ordered, but then it takes less time to search in it! (It takes time to tidy up your room, but then you find your things faster!)

Heap sort is based on the data structure known as *complete binary heap*. This is a data structure that allows us

- to insert multiple elements ($\Theta(\log n)$ time for inserting a single element where n is the current number of elements in the heap),
- to quickly find the current largest element (in time $O(1)$), and
- to remove the current largest element (in time $\Theta(\log n)$).

This is perfect for sorting—we first add all the elements to the heap, then we repeatedly extract (i.e., find and then remove) the greatest element in the heap and write it to our output. If n is the number of elements, we perform in total $O(n)$ heap operations, each with a running time not larger than $O(\log n)$ (note that the current number of elements is always bounded by n). Thus we see that heap sort runs in $O(n \log n)$ time. Note that our time analysis yielded only an upper bound in "O"-notation. So, is the actual running time better? The answer is no! As the heap operations are solely based on comparisons between the elements (as we will see), heap sort cannot be faster than the lower bound of $\Omega(n \log n)$ from our theorem above. Consequently, the actual running time of heap sort is $\Theta(n \log n)$.

A complete binary heap is represented as a subarray $a[1..i]$ of a possible larger array $a[1..n]$. (In most programming languages, including Python and C++, array indices start with 0; however, it is a bit easier to describe the complete binary heap starting with index 1. In practice, we can just add a dummy [0] element and not use it, or add -1 on all the indices.) We arrange the elements into a tree-like structure, where elements $2k$ and $2k+1$ are called the children of k , and, conversely, $\lfloor k/2 \rfloor$ is called the parent of k . The heap has to satisfy the invariant called the *heap property*.

Heap Property: *the value in the parent is never smaller than the values in its children.*

As an example of the tree-like structure, see the slides and the following [visualization](#). The visualization (which shows how the heap evolves during heap sort) will be explained later in more detail. For now, just consider the particular heaps independently of each other and ignore the red and blue strokes. The elements belonging to the heaps are white, the other ones are gray. The first heap is empty and therefore all elements are gray. The index of an element corresponds to its index in the array. Thus, the index of the last white element (if there is a white element) is the before-mentioned index i and corresponds to the current number of elements in the heap. Observe that that the white elements always satisfy the heap property. **You can play around and change the sequence of values above!**

Regarding the name *complete binary heap*: "binary" refers to the property that every element has at most two spots for its children, and "complete" refers to the shape where all the possible children spots on every row are filled with the sole exception of the last row. On the last row, the filled children spots form a consecutive sequence starting at the left-most spot.

The heap property ensures that the currently largest element is always $a[1]$. To add an element to the heap (with i elements), we first put it at the last position (as $a[i+1]$), then we move it upwards the tree as long as it is greater than its parent. This takes at most $O(\log i)$ steps (one can show this bound using the property that the heap is a complete binary tree), which again is bounded by $O(\log n)$ (as $i \leq n$). The following Python code shows such a "upheap" function that moves the element $a[i]$ upwards to its correct position.

```
upheap(a, i):
    while i>1 and a[i//2] < a[i]:
        # The operation (a, b) = (b, a)
        # swaps the values of a and b
        (a[i], a[i//2]) = (a[i//2], a[i])
        i = i//2
```

The first part of the [visualization](#) shows, for each element, how the heap looks like after the element has been added. The red stroke marks all the spots in the heap that have changed after the insertion.

To remove the largest element from the heap, we switch its location with the very last element ($a[i]$) and then remove it. (In fact, we don't have to swap both elements; it suffices to just overwrite $a[1]$ with the value of $a[i]$. However, the swapping will be useful for sorting.) After this, the new element $a[1]$ is probably not placed correctly, so we move it downwards to its correct place (similarly to the upheap function, but moving downwards instead of upwards). The following Python code shows such a "downheap" operation that moves the element $a[i]$ downwards to its correct position.

```
downheap(a,i,n):
    while true:
        # We check
        #     if the left child exists and
        #     its value is greater than a[i]
        go_left = 2*i<=n and a[2*i] > a[i]
        # Same for right
        go_right = (2*i+1) <= n and a[2*i+1] > a[i]
        # If we can go left and right,
        #     we want to go to the child with higher value.
        if go_left and go_right:
            if a[2*i] > a[2*i+1]:
                go_right = False
            else:
                go_left = False
        # And here we go either to left or to right (if we can)
        if go_left:
            (a[i], a[2*i]) = (a[2*i], a[i])
            i = 2*i
        elif go_right:
            (a[i], a[2*i+1]) = (a[2*i+1], a[i])
            i = 2*i+1
        else:
            return
```

In the second part of the [visualization](#), we iteratively remove the maximum element until the heap becomes empty. The blue strokes show where the last maximum has been placed. The red strokes mark again the spots that changed in the heap. Observe that at the end, the array is sorted!

As the visualization suggests, it is possible to implement heap sort using only $O(1)$ extra memory. At any moment, where our heap has i elements, we will use the subarray $a[1..i]$ of our input array $a[1..n]$ to represent our heap. Initially, we have $i=0$ and $a[1..0]$ is an empty heap. To insert the $(i+1)$ th element, we just run $\text{upheap}(a, i+1)$ and then increase i by one. Thus, during the first phase (storing all the elements in the heap), after i steps, the first i elements are in the heap, and the remaining elements have not yet been added. During the second phase (moving the elements from the heap back to the array in the correct order), after j steps, the heap contains only $n-j$ elements (hence, occupies $a[1..n-j]$), whereas the remaining j elements ($a[n-i+1..n]$) are the largest elements of a and correctly sorted. The whole algorithm heap sort is implemented as follows:

```
for i in range(1, n+1):
    upheap(a, i)
for i in range(n, 1, -1):
    (a[1], a[i]) = (a[i], a[1])
    downheap(a, 1, i-1)
```

Note that it is actually possible to construct a heap in just $O(n)$ time by appropriately using downheap instead of upheap . Such a construction is therefore more efficient than what we do above in the first phase of our algorithm (that takes $\Theta(n \log n)$). Notwithstanding, such a speed-up does not affect the overall asymptotic running time of heap sort as the second phase still takes $\Theta(n \log n)$ time in the worst case. Concerning the upheap function, it is thus not really needed for heap sort. However, it will be useful for us in a later chapter (Lecture 8).

All in all, the time worst-case complexity of heap sort is always $\Theta(n \log n)$ and the memory complexity is $O(1)$, which is very good. The drawback is that heap sort is not stable, though. It is a bit faster than merge sort in practice, but the lack of stability may be an issue in some situations.

Interestingly, it took about 30 years from the invention of heap sort until its best-case running time had been settled. Although, formally, the best case is trivially $\Theta(n)$ if all elements are equal, in the more realistic and practical setting where all elements are pairwise distinct, the best-case running time [turned out](#) to be the same as the worst case: $\Theta(n \log n)$.

Sorting in Real Life

What sorting algorithms are used in practice? Popular programming languages today have built-in sorting functions. They are based on the rough ideas given above, but they use powerful optimizations, which make them even better. We will give a rough overview of the implementations in C++ and in Python.

C++

In C++, two sorting functions are available in the standard library: `sort` and `stable_sort`. Stability is an important property in many applications, so we often want to use a stable algorithm; most popular implementations of C++ use merge sort for `stable_sort`. However, it takes extra resources to guarantee stability, so if stability is not necessary, it is better to use an unstable algorithm such as `sort`.

The fastest algorithm in practice is quick sort, and so, not surprisingly, the implementation of `sort` is based on quick sort (with a smart method of choosing the pivot). However, quick sort has two disadvantages:

- When sorting a very small number of elements, insertion sort is actually the fastest of the mentioned sorting algorithms. However, we can easily combine the advantages of quick sort and insertion sort to get something better than both of them: we recursively use quick sort as long as the current (sub)array that we want to sort is not sufficiently small. If the subarray is sufficiently small, we switch to insertion sort in order to sort it!
- There are bad cases for quick sort that take $\Omega(n^2)$ time. This problem can be mitigated by using so-called introspection: the algorithm studies how well it is doing, and if it finds out that it is not doing very well (bad pivots are used, causing the next iteration to sort almost all the elements more often than expected), we switch to another sorting algorithm. A good candidate for such another sorting algorithm is heap sort (as it is more efficient than merge sort). This approach is known as introsort.

A hybrid of these two approaches is internally used in implementations of the C++ `sort` function.

Python

For simplicity, Python has only one sorting function. It is stable and, similar to `stable_sort` in C++, its general idea is based on the best stable sorting algorithm covered in these notes: merge sort. However, the idea is further improved by insights coming from other algorithms.

We have mentioned that insertion sort works very quickly (in $O(n)$) on data that is already sorted. Similarly, it is easy to construct an algorithm that works very quickly on data that is in reverse order. All the other algorithms mentioned above (quick sort, merge sort, heap sort) take at least $\Omega(n \log n)$ time in the best case.

The sorting algorithm implemented in most versions of Python, called timsort, combines the advantages of merge sort and insertion sort. We first look for 'runs' in our data: these are segments which are already sorted, or which are reversely sorted. The reverse sorted segments are reversed, and afterwards, all the runs are merged, using merge sort.

This yields a sorting algorithm which is not only $O(n \log n)$ in the worst case, but also $O(n)$ in many "easy" cases. Since these "easy" cases actually occur quite often in real world data, timsort is very fast in practice.

Can We Sort Even Better?

The last section may give an impression that it is pointless to implement sorting algorithms of our own—after all, our programming languages use algorithms which are more sophisticated than what we could probably implement ourselves, and also more efficient. We have also proven a theorem saying that we cannot sort data faster than in $O(n \log n)$.

However, this is not the case—there are situations when a simple sorting algorithm actually works better! This is because our theorem makes only a statement on sorting algorithms based on comparisons. If we know that our data to be sorted has some extra structure, it may be possible to sort it faster than in $O(n \log n)$ time, and also much faster than the built-in implementations!

Counting Sort

We have shown the techniques of counting and cumulative sums in the previous lectures, but we have not yet tried to use them for sorting. Imagine that you are a teacher, just after grading the tests (let's say there are 200 of them), and you want to sort the test sheets alphabetically by the students' last names, so that you can find them easily when the students complain. What is the quickest way to accomplish this?

A method that turns out to be very good in practice is not based on any of the sorting algorithms above. It works as follows: we create 26 piles of tests, each for a different letter of the alphabet (A..Z). Then we look at every test, and we drop it on the pile corresponding to the student's name.

Then we simply collect all the piles in order. Of course this still leaves students not sorted correctly if they share the first letter of their names. Let's assume for now that it is good enough: every student is close to the expected position, and we can easily find them.

Counting sort is an algorithm based on the idea above. It is used if there is a small number (k) of possible keys. For example, when we are sorting 10000 people born in the last century by their birth year, there are only 100 different keys (1900 to 1999), which is a small number of possibilities. The idea is first to simply count the number of people born in each year. Then, for every key, we compute the first position in our output array that will contain a person with this key (using cumulative sums), and then place each person on its correct location.

For example, assume that we have the following people A (1907), B (1905), C (1907), D (1909). We count that there is 1 person born in 1905, 2

persons born in 1907, 1 person born in 1909. By using cumulative sums, we find out that people born in 1905 should start at position 0, people born in 1907 should start at position 1, and people born in 1909 should start at position 3. We look through our list again and move all the people to their correct positions (A is moved to 1, B is moved to 0, C is moved to 2 because 1 was already taken, D is moved to 3).

The time complexity of counting sort is $\Theta(n+k)$, the memory complexity also $\Theta(n+k)$. This sorting algorithm is stable. This algorithm is not based on comparisons, which allows it to circumvent the theorem mentioned above, and to run faster than $\Omega(n \log n)$ (if k is sufficiently small). As the time complexity suggests, this can work noticeable faster than the standard library algorithms—but it needs specific kind of data to work. However, it is often the case that the number of possible keys is small.

On the slides, you find the algorithm in pseudo code.

Radix Sort

The counting sort examples given above are not completely satisfying. In the example with the students, we still had to sort every pile alphabetically by the remaining letters. In the birth year example, we have sorted people by their birth year, but what about month and day?

One way of solving this issue is to use radix sort. We explain it on the birth year example above:

- First, we sort all the people by day (1..31), using counting sort. We ignore the month and year for now.
- Next, we sort all the people by month (1..12), again using counting sort. We do this in a stable way.
- Finally, we sort all the people by year (0..99), using counting sort again. We do this in a stable way.

Since we were using a **stable** sorting algorithm in the last phase, people in every year will remain sorted with respect to month. Likewise, for every month and year, people will be correctly sorted with respect to the day. Therefore all the people are sorted correctly, and we did it quite quickly, in time $\Theta(n+31)+\Theta(n+12)+\Theta(n+100)=\Theta(n)$! A similar method could be also used for alphabetical sorting, although it is more difficult there, because the names could be long and have different lengths.

Interestingly to know (though not obligatory for our course): radix sort can be used to sort n numbers in time $O(n)$ even if the are as large as some polynomial in n (that is, the maximum key size $k=nO(1)$). Counting Sort alone would yield a bad polynomial running time as $O(n+k)=nO(n)$ in such a case. However, if we split the keys into into constant many chunks of size n (which is possible in this case), and run radix sort on them (that is, we run

counting sort constant many times), we have sorted the numbers in linear time!

Both counting sort and radix sort are easy to implement, and they often can be used to sort data much quicker than the built-in sorting functions.

Ostatnia modyfikacja: wtorek, 2 czerwca 2026, 22:19

[◀ Lecture 7 - recording](#)

Przejdź do...

[Lecture 8 ▶](#)

 Skontaktuj się z pomocą techniczną

Jesteś zalogowany(a) jako Ondřej Marvan (Wyloguj)

2400-DS1AL#2025L#277553